

Smart HR

사내 문서 기반 LLM 질의응답 웹 애플리케이션

SMART 2팀
팀장 : 최상욱
팀원 : 박상희 신누리 소성민

목차

01 프로젝트 개요

03 LLM & RAG

02 Django 마이그레이션

04 AWS 배포

프로젝트 개요

사내 인사·총무 문의를 RAG 기반 LLM 챗봇으로 즉시 해결

배경

취업규칙·복무규정·인사규정 등 내부 문서를 임직원이 직접 찾아보지 않고도, 자연어 질의만으로 즉시 답변받을 수 있는 사내 서비스

과업

- 요구사항 분석 및 화면 설계
- HTML·CSS·JS + Django 기반 Web Application 구축
- LLM 질의응답 + RAG(문서 검색·근거 제시) 기능 통합
- 클라우드 인프라 구성 및 배포

핵심 가치

- **즉시성**
규정을 찾아 헤매지 않고 질문 즉시 답변
- **신뢰성**
근거 문서·조항을 답변과 함께 제시
- **운영 효율**
반복 문의로 인한 인사팀 업무 부담 절감
- **확장성**
Django 전환으로 유지보수·확장 용이

기능 요구사항 요약

인증 / 회원 관리

로그인·회원가입·아이디 중복확인·로그아웃·세션 만료·진입 라우팅

챗봇 (RAG 질의응답)

대화방 CRUD·질의응답·근거문서 표시·대화방명 자동생성·오류처리

관리자 · 사용자 관리

목록조회·추가·수정·삭제(소프트 삭제)

관리자 · RAG 문서관리

업로드·일괄 적재·미리보기·문서 API 연동정보 제공

관리자 · 통계

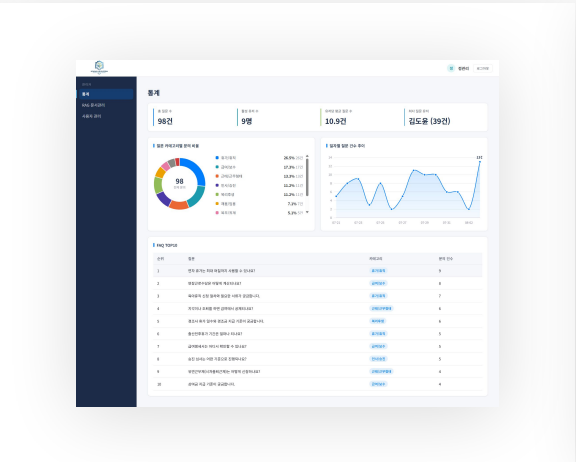
요약지표·카테고리 비율·일자별 추이·FAQ TOP10

주요 화면



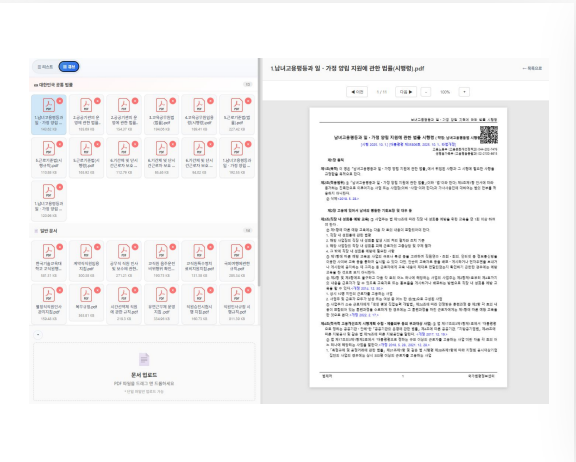
chat

AI 인사 상담 페이지 (직원용)
인사/복지후생 규정 질의응답,
법령 기반 답변, 근거 문서
출처 제공,맞춤형 HR 안내



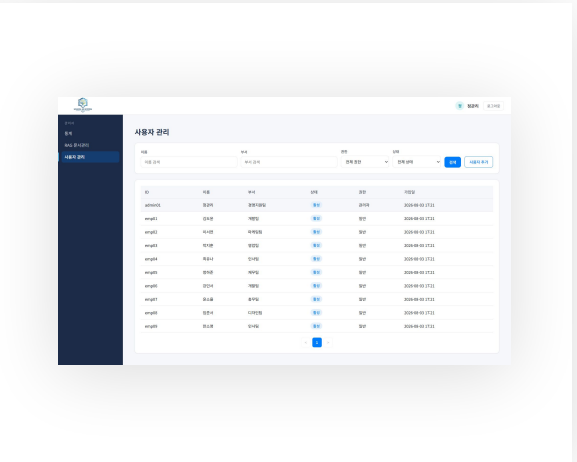
Dashboard

분석 대시보드 페이지
(관리자용)
질의 유형 분석, 자주 묻는 질문
TOP10,부서별 문의 현황, AI
상담 이용 통계



문서관리

규정 문서 관리 페이지 (관리자용)
취업규칙 및 교내 규정 업로드,
문서 임베딩 및 Vector DB 저장,
규정 업데이트 관리

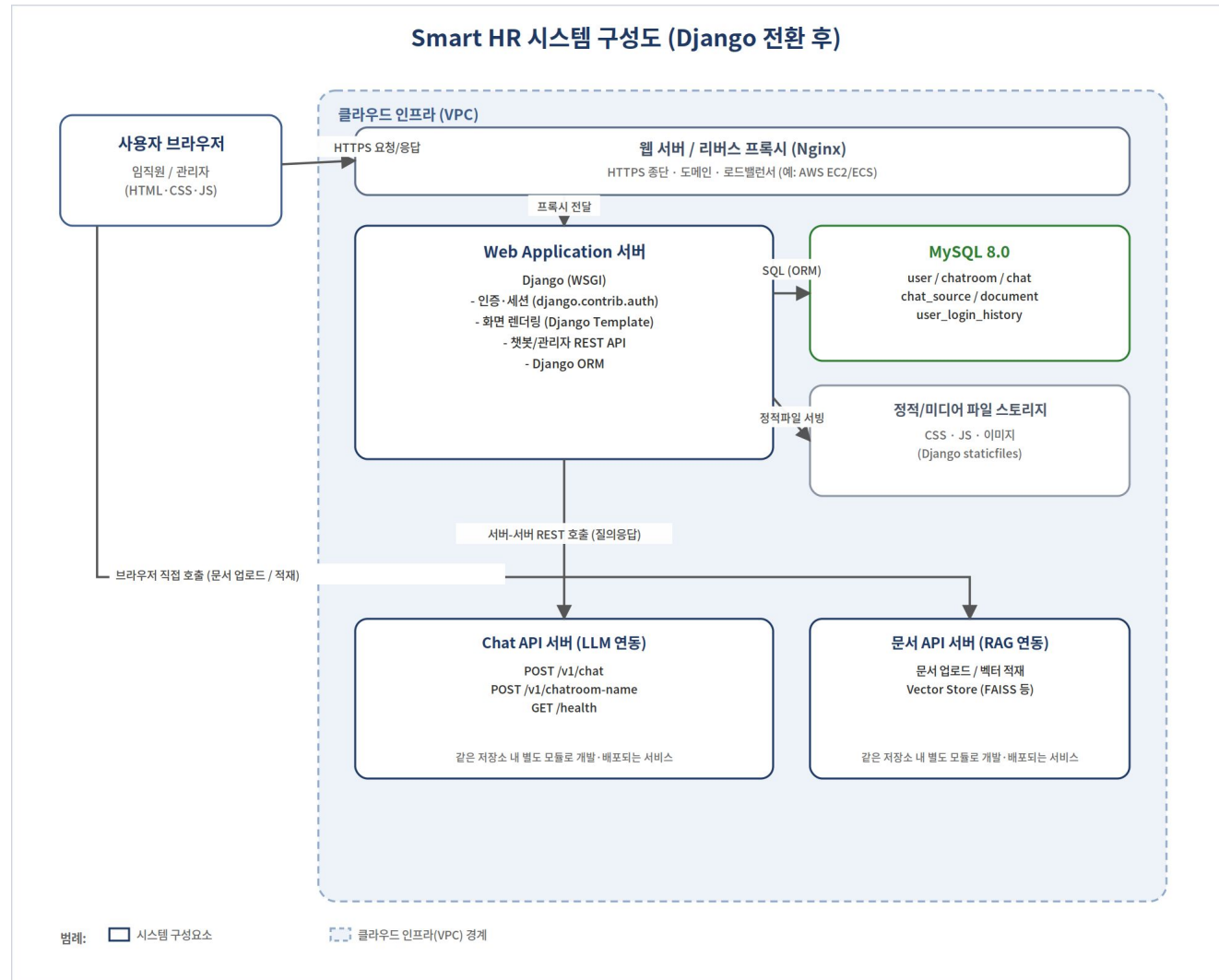


유저관리

유저 관리 페이지 (관리자용)
(유저 권한 및 정보 수정/삭제)

시스템 아키텍처

Django Web Application + Chat API(LLM) + 문서 API(RAG)



데이터 확보 경로 및 전처리

데이터 확보 경로

- 국가법령정보센터 등 공식 출처의 법령 PDF (근로기준법, 남녀고용평등법, 공공기관 운영법, 교육공무원법 등)
- 사내 규정 PDF (복무규정, 인사규정 등)
- "공통 법률" / "일반 문서"로 구분해 문서 API의 document 테이블에 메타데이터로 등록
- 총 26건의 문서로 초기 코퍼스 구성, 관리자 화면에서 업로드로 확장 가능

전처리 파이프라인

- 페이지 텍스트 공백/개행 정규화
- 조문 시작("제N조") 경계를 최우선 구분자로 두어 조문이 잘리지 않도록 청킹
- 페이지 경계를 넘어 이어지는 조문은 이전 페이지의 조문 헤더를 이어붙여 보강
- 청크 크기 400자 내외, 오버랩 80~120자로 문맥 손실 최소화
- 적재 시 문서별 FAISS 인덱스 (vector_store/<doc_id>)로 분리 저장 — 문서 단위 재적재·삭제 용이

DB 설계 — 핵심 테이블 (Django 전환 반영)

MySQL 8.0 · Django ORM 마이그레이션 기준 · 7개 테이블 (documents 앱 신규 편입)

테이블	주요 컬럼	설명
user	username(PK, db_column=user_id), password, name, department, is_admin	사용자 계정 — Django AbstractBaseUser 기반
user_login_history	history_id(PK), user_id(FK), created_at	로그인 이력
chatroom	chatroom_id(PK, UUID), user_id(FK), chatroom_name	대화방
chat	chat_id(PK), chatroom_id(FK), speaker, message, topic	대화 (질의/답변)
chat_source	source_id(PK), chat_id(FK), doc_id, file_name, document_title★, article★, page	답변 근거 문서 스냅샷 — ★신규 컬럼
document	doc_id(PK), original_file_name, is_loaded	RAG 대상 업로드 문서
auth 권한 테이블	user_groups, user_user_permissions	Django PermissionsMixin이 자동 생성 (M2M)

- documents 테이블이 web/documents/ Django 앱으로 편입 — 더 이상 "RAG 서비스 전용 메타데이터"가 아니라 Django ORM이 직접 관리 (마이그레이션·인덱스 포함)
- chat_source에 document_title·article 컬럼 추가 — RAG가 검증한 근거의 문서명·조항을 채팅 이력에도 그대로 보존
- FK 삭제 정책이 전부 ON DELETE CASCADE로 통일(Django 기본값) — 애플리케이션의 소프트 삭제(is_deleted)와는 별개로, DB 레벨에서는 부모 삭제 시 자식이 함께 삭제되는 정책으로 변경됨

DB 스키마 상세

테이블	컬럼	타입	제약 / 키
user	username / password / name / department / is_admin·is_disabled·is_deleted / is_superuser·last_login	VARCHAR(20) / VARCHAR(128) / VARCHAR(50) / VARCHAR(100) / BOOLEAN / BOOLEAN·DATETIME	PK(username, db_column=user_id) · password는 Django 표준 필드(bcrypt로 재정의)
user_login_history	history_id / user_id / created_at	INT AI / VARCHAR(20) / DATETIME	PK(history_id), FK→user (CASCADE)
chatroom	chatroom_id / user_id / chatroom_name / is_deleted	CHAR(36) UUID / VARCHAR(20) / VARCHAR(100) / BOOLEAN	PK(chatroom_id), FK→user (CASCADE)
chat	chat_id / chatroom_id / speaker / message / topic	INT AI / CHAR(36) / VARCHAR(4) / TEXT / VARCHAR(100)	PK(chat_id), FK→chatroom (CASCADE) · speaker는 CharField+choices(user/llm), MySQL ENUM 아님
chat_source	source_id / chat_id / doc_id / file_name / document_title★ / article★ / page	INT AI / INT / INT NULL / VARCHAR(255) / VARCHAR(255) NULL★ / VARCHAR(255) NULL★ / INT NULL	PK(source_id), FK→chat (CASCADE) · doc_id는 약한 참조 · ★ 0003 마이그레이션에서 추가
document	doc_id / original_file_name / stored_file_name / is_loaded	INT AI / VARCHAR(255) / VARCHAR(255) / BOOLEAN	PK(doc_id) · web/documents 앱, 복합 인덱스 (is_deleted, created_at)

Legacy DB 안전 이관

FastAPI · SQLAlchemy DB → Django migration 스키마

왜 이관이 필요한가 새 Django 앱은 **migrate**가 만든 새 스키마 DB에서만 동작한다. 이전 DB를 그대로 연결하면 **Table 'user' already exists** 로 마이그레이션이 실패하고, 새로 만든 빈 DB에는 기존 사용자·대화·이력이 하나도 없다.

스키마가 달라 이전 DB를 그대로 못 쓴다

항목	이전 DB	새 Django
User 인증	passwd	password · last_login · is_superuser · 권한 테이블
ORM / 스키마	SQLAlchemy	Django ORM (migrate가 생성)
chat_source	기본 컬럼	+ document_title · article (legacy 행은 null)

이관 방식 — **import_legacy_data** 관리 명령



이관 대상 (5개) user · user_login_history · chatroom · chat · chat_source
제외 (RAG 새로 시작) document · PDF · vector_store — chat_source는 file_name·page 스냅샷만 보존

결론 두 스키마가 같은 테이블명 **user**를 써서 충돌 → 새 빈 DB에 migrate 후 dry-run 검증을 거쳐 이관한다.

안전장치 — 되돌릴 수 없는 실수를 코드로 차단

Dry-run 기본
--apply 로만 실제 기록

FK 사전검증
부모 없는 자식 행이면 쓰기 전 중단

대상 비어있음 검사
기본 모드에서 데이터가 있으면 중단

트랜잭션 원자성
전부 성공 또는 전부 취소

bcrypt 해시 보존
재해싱 안 함 → 로그인 유지

타임스탬프 복원
생성·수정 시각 원본 유지

Django 마이그레이션

파트: FastAPI프로젝트의 Django 마이그레이션

확인사항 (계속) 1/2

view

대시보드

스토리보드

view

대시보드

스토리보드

view

views.member.addMember.jsp

로그인

view

유저관리

유저관리

view

유저관리

유저관리

view

views.member.addMember.jsp

로그인

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

view

유저관리

유저관리

확인사항 – 이전 작업내역

기본 프로젝트는 FastAPI, SQLAlchemy, MySQL, Jinja2 Server Side Rendering, JavaScript, CSS를 이용

Web파트는 port 8000, users auth, chat, 대화내역관리, admin functions, 정적파일, 문서관리를 확인 할 수 있었다.

* SQLAlchemy: Python에서 DB를 다루기 위한 ORM/SQL Toolkit

* Jinja2 Server-Side Rendering: HTML에 Python 데이터를 끼워 넣어서 HTML을 만들어주는 템플릿 엔진

* Server-Side Rendering(SSR): HTML을 서버에서 완성해서 브라우저로 보내는 것

확인사항 – 구조 (이전에 맡은 파트 RAG 제외)

1. FastAPI backend, SQLAlchemy ORM, MySQL database, Jinja2 server-side rendering, Authentication with session middleware, Jinja2 템플릿, Static files (css, js, img) 전체 구조 및 프로세스 분석
2. 외 무엇을 router하는가: auth, chat, admin(users, documents, stats)
3. 서비스와의 연결 user, chat, llm_client
4. 모델은 어떻게 구성되어있는지: user, chat

Django 마이그레이션

파트: FastAPI프로젝트의 Django 마이그레이션

확인사항 (완료) 1/2

```

Django
├── Django ORM
├── Django Admin
├── Django Migration
└── SQLAlchemy ← 별도 ORM
  
```

```

Django
├── Models
├── ORM
├── Migration
├── Admin
└── DB(MySQL)
  
```

```

FastAPI
├── Session Middleware
│   └── 직접 만든 인증 로직
│       └── User
  
```

```

Django
├── Django Authentication
│   ├── User
│   ├── Session
│   ├── Login / Logout
│   └── 권한/Permission
  
```

확인사항 – DB

SQLAlchemy, DB 모델을 Python 객체로 관리하는 역할

Django 프로젝트에서 계속 SQLAlchemy를 사용할 수도 있다.

다만 그러면 구조가 애매해진다. DB 접근 계층이 이중화.

Django는 자체적으로 ORM, Migration, Admin, Authentication, Session, Form, Template 을 제공해 ORM으로 통일하면 구조가 훨씬 단순해진다. 프레임워크를 FastAPI → Django로 바꾸는 것이 프로젝트의 목적이니 기존 SQLAlchemy 모델을 **Django ORM 모델**로 다시 작성해보자.

DB는 그대로 MySQL유지, Python에서 DB를 다루는 ORM 계층만 Django ORM으로 교체로 결정.

확인사항 – user 인증

기존 프로젝트의 구조는 인증을 **직접 관리**했기 때문에

로그인 – 사용자 확인 – 세션생성 – 세션 user 정보 저장 – 각 api에서 세션 확인 – 로그아웃 시 세션 삭제를 관리한다. Django는 이 중 **User / Authentication / Session / Admin / Permission**을 상당 부분 기본 제공되어 이미 인증 시스템이 프레임워크에 통합되어 있다.

'확인사항 – DB' 때와 마찬가지로 Django로 옮기면서 기존 인증을 그대로 가져가기보다 Django로 이전하면서 인증 시스템도 프로젝트 의도에 맞게 Django 방식으로 통일하기로 결정.

Django 마이그레이션

파트: FastAPI 프로젝트의 Django 마이그레이션

수행계획

1. 현재 FastAPI 웹 구조(모델, 라우터, 서비스, 템플릿) **분석**
2. **Django** 프로젝트 구조 및 설정 생성
3. **데이터베이스** 모델을 SQLAlchemy에서 Django ORM으로 마이그레이션
4. **인증 시스템**을 Django 인증(auth)으로 마이그레이션
5. **뷰/라우터**를 FastAPI에서 Django 뷰로 마이그레이션
6. **템플릿**을 Jinja2에서 Django 템플릿으로 마이그레이션
7. **정적 파일** 마이그레이션 및 CSS/JS 참조 업데이트
8. Django 의존성에 맞춰 **requirements.txt** 업데이트
9. **설정** 및 **환경 변수** 업데이트
10. 마이그레이션된 Django 애플리케이션 **테스트**

Django 마이그레이션

파트: FastAPI프로젝트의 Django 마이그레이션

수행 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

1. 현재 FastAPI 웹 구조(모델, 라우터, 서비스, 템플릿) 분석
2. Django 프로젝트 구조 및 설정 생성
8. Django 의존성에 맞춰 requirements.txt 업데이트

```
django-admin startproject config .  
python manage.py startapp users  
python manage.py startapp chat
```

```
fastapi - Update - to Django  
sqlalchemy - Update - to Django ORM  
pymysql - Update - to Django mysqlclient  
uvicorn - Delete - to Django Gunicorn/ASGI..  
jinja2 - Delete - to Django Template  
python-multipart - Delete - Django에서 필요없음.  
itsdangerous - Delete - Django에서 기본 세션/서명 체계 기능을 불필요  
pip install -r  
"d:\study\sk_playdata\personal\SKN32-4th-2Team\web\requirements.txt"
```

Django 마이그레이션

파트: FastAPI 프로젝트의 Django 마이그레이션

수행 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

9. 설정 및 환경 변수 업데이트

Django가 자동 생성한 기본 settings.py에

- env 설정 추가: load_dotenv(), os.getenv()
- ALLOWED_HOSTS = [] * 추가, 개발용 전환
- INSTALLED_APPS에 users, chat 추가
- template 경로 추가 "DIRS": [BASE_DIR / "templates"]
- static 설정: STATIC_URL, STATICFILES_DIRS, STATIC_ROOT
- media 설정(사용자 업로드 파일용): MEDIA_URL = "/media/"
- session 설정: getenv("SESSION_MAX_AGE_SECONDS", str(3 * 60 * 60))
- user model 설정: AUTH_USER_MODEL = "users.User", Django Authentication의 users.User로 **모델만 교체해** 인증시스템을 사용.
- api 설정: CHAT_API_BASE_URL, CHAT_API_TIMEOUT_SECONDS(default 15), DOC_API_BASE_URL
- 외 기본값 수정: LANGUAGE_CODE = "ko-kr", TIME_ZONE = "Asia/Seoul"

```
DATABASE_URL=mysql+pymysql://change-user:change-pw@change-ip:change-port/rag_chatbot?charset=utf8mb4
```

```
SESSION_SECRET_KEY=change-me-to-a-random-secret  
CHAT_API_BASE_URL=http://localhost:8002  
CHAT_API_TIMEOUT_SECONDS=15  
DOC_API_BASE_URL=http://localhost:8001
```

*env

Django 마이그레이션

파트: FastAPI 프로젝트의 Django 마이그레이션

수행 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

3. 데이터베이스 모델을 SQLAlchemy에서 Django ORM으로 마이그레이션 (기존 사용자 채팅 기능의 구조와 데이터 호환성을 유지)

```
class User(Base):  
    class User(AbstractBaseUser, PermissionsMixin):  
  
        name = Column(String(50), nullable=False)  
        username = models.CharField(max_length=20,  
                                    primary_key=True, db_column='user_id')  
  
    class UserManager(BaseUserManager):  
  
    class Chatroom(Base):  
        class Chatroom(Base):
```

models.Model을 기반으로 Django ORM을 적용하되 기존 User 모델의 커스터마이징을 위해 `AbstractBaseUser`를 상속

Django의 권한 및 그룹 관리 기능을 활용하기 위해 `PermissionsMixin`을 추가

기존 SQLAlchemy의 `Column(String())`, `Column(Integer())` 등의 데이터 타입을 `Django ORM`의 `models.CharField()`, `models.IntegerField()` 등으로 매핑해 전환.

기존에 사용하던 User 생성 방식을

Django가 요구하는 Custom User Model 방식에 맞춰 `UserManager`로 구현

채팅 모델도 기존 SQLAlchemy 기반 `Chatroom(Base)`를 Django ORM 기반 `Chatroom(models.Model)`으로 전환

기존 Chatroom의 데이터 구조와 기능을 Django ORM에서 사용할 수 있도록 변경

Django 마이그레이션

파트: FastAPI프로젝트의 Django 마이그레이션

수행 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

4. 인증 시스템을 Django 인증(auth)으로 마이그레이션

```
bcrypt.hashpw(raw_password.encode("utf-8"), bcrypt.gensalt())
bcrypt.hashpw(raw_password.encode("utf-8"), bcrypt.gensalt()).decode('utf-8')

@router.get("/api/list")    path('api/list', admin_views
```

인증 영역에서는 기존 시스템에서 사용하던**bcrypt기반 비밀번호 해싱 방식을 유지**하여 기존 사용자 비밀번호 처리 방식과의 호환성을 확보
User 조회는 Django ORM으로 전환한 뒤 인증 성공 시 Django의 login()을 통해 Session을 관리하도록 변경

기존 FastAPI에서는 **@router.get()**과 같이 라우터 **데코레이터**를 통해 URL, HTTP Method, 처리 함수를 함께 정의했으나,
Django에서는 요청 처리를 담당하는**View**와 **URL** 라우팅 설정을 분리

(views.py에서 users_page(request) View를 구현해 urls.py에서 path("admin/users", users_page)를 통해 해당 URL과 View를 매핑)

Django 마이그레이션

파트: FastAPI프로젝트의 Django 마이그레이션

수행 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

5. 뷰/라우터를 FastAPI에서 Django 뷰로 마이그레이션

기존 구조: 사용자 → FastAPI Web → Chat API /v1/chat → LLM + RAG → 답변

변경된 구조: 사용자 → Django → chat/services.py → 기존 Chat API → LLM + RAG → 답변

FastAPI: @router.post()

Django: def view(request) + urls.py의 path()

Chat/LLM 자체는 Django로 옮기지 않고, Django가 기존 AI 서버를 호출할 수 있도록 중간 연결 코드(client/service)를 생성해 Web 계층만 Django로 마이그레이션하고,
AI/LLM/RAG 서버는 httpx와 호출 대상인 기존 Chat API는 그대로 사용.
(기존 FastAPI 기반 API로 유지하여 서비스 간 HTTP 통신으로 연결)

Django 마이그레이션

파트: FastAPI 프로젝트의 Django 마이그레이션

수행 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

6. 템플릿을 Jinja2에서 Django 템플릿으로 마이그레이션

기존 Jinja2 템플릿을 Django 템플릿으로 변경

TemplateSyntaxError:

sidebar.html

Could not parse the remainder: ' or ''' from 'chatroom_id or '''

Jinja2에서는 가능했던 표현을 Django Template에서는 그대로 못쓰는 문제 발생, sidebar.html 조건문 변경 및 해결
불필요한 ' ' 부분 삭제하거나 or ' '부분을{% endif %}로 분리해 처리.

```
{% if chatroom_id == current_chatroom_id or chatroom_id == '' %}
```

```
{% if chatroom_id == current_chatroom_id %}
```

마찬가지로 Jinja 스타일의 문법이 Django Template에서 사용되고 있는 부분을 중점적으로 점검

– {{ variable[:1] }} 같은 Python slicing

header.html의 {{ user.name[:1] }}를 <div class="avatar">{{ user.name|slice:"1" }}</div>

– 조건식 내부의 Python 문법, for/if 등의 Jinja2와 Django Template 문법 차이, Jinja2에서 지원하지만 Django Template에서는 지원하지 않는 문법

sidebar.html의 {{ 'active' if active == 'chat_new' }}

– 템플릿에서 직접 실행되는 Python 표현식

<form method="post" action="/auth/logout">를 <form method="post" action="{% url 'logout_submit' %}"> {% csrf_token %}

Django 마이그레이션

파트: FastAPI 프로젝트의 Django 마이그레이션

수행 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

7. 정적 파일 마이그레이션 및 CSS/JS 참조 업데이트

(앞서 확인한 settings.py) – static 설정: `STATIC_URL`, `STATICFILES_DIRS`, `STATIC_ROOT`

FastAPI에서 StaticFiles를 통해 관리하던 정적 파일 제공 방식을 Django의 staticfiles 시스템으로 전환

기존 템플릿의 정적 파일 경로를 Django 템플릿의 `{% load static %}`, `{% static %}` 방식으로 변경

= 기존 FastAPI main.py에서 `app.mount("/static", StaticFiles(...))` /static URL

`<link rel="stylesheet" href="res/css/rag.css">` `<script src="https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.11.174/pdf.min.js"></script>`

settings.py의 `STATIC_URL` 설정과 `{% load static %}`

`<link rel="stylesheet" href="{% static 'css/style.css' %}">` `<script src="{% static 'js/app.js' %}"></script>`와 같이 변경

Django가 문서 처리 로직을 새로 구현한 것이 아니라, 기존 Document API를 호출하는 Web 계층으로 동작하도록 변경

Django에서는 POST/PATCH/DELETE 같은 요청에 `CSRF 검증`이 필요함으로 JavaScript의 POST 요청에 X-CSRFToken Header 추가해

브라우저에 저장된 Django CSRF Cookie에서 Token을 꺼내 HTTP Header에 넣어 Django 서버에 전달

`headers: {'X-CSRFToken': getCookie('csrftoken')}`

Django 마이그레이션

파트: FastAPI프로젝트의 Django 마이그레이션

수행 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

10. 마이그레이션된 Django 애플리케이션 테스트

```
python manage.py migrate
k_playdata\personal\WSKN32-4th-2Team\Wweb"; python
manage.py micd
"d:\Wstudy\Wsk_playdata\personal\WSKN32-4th-2Team\Ww
eb"; python manage.py micd
"d:\Wstudy\Wsk_playdata\personal\WSKN32-4th-2Team\Ww
eb"; python manage.py micd
"d:\Wstudy\Wsk_playdata\personal\WSKN32-4th-2Team\Ww
eb"; python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, chat, contenttypes,
sessions, users
Running migrations:
  Applying contenttypes.0001_initial ... OK
  Applying
contenttypes.0002_remove_content_type_name ... OK
  Applying auth.0001_initial... OK
  Applying auth.0002_alter_permission_name_max_length ...
OK
  Applying auth.0003_alter_user_email_max_length ... OK
  Applying auth.0004_alter_user_username_opts ... OK
  Applying auth.0005_alter_user_last_login_null ... OK
  Applying auth.0006_require_contenttypes_0002 ... OK
  Applying
auth.0007_alter_validators_add_error_messages ... OK
  Applying auth.0008_alter_user_username_max_length ...
OK
  Applying auth.0009_alter_user_last_name_max_length ...
OK
  Applying auth.0010_alter_group_name_max_length ... OK
  Applying auth.0011_update_proxy_permissions ... OK
  Applying auth.0012_alter_user_first_name_max_length ...
OK
  Applying users.0001_initial ... OK
  Applying admin.0001_initial ... OK
  Applying admin.0002_logentry_remove_auto_add ... OK
  Applying admin.0003_logentry_add_action_flag_choices ...
```

uvicorn.run()을 Django방식 execute_from_command_line([])로 변경해
기존 Uvicorn를 거치던 FastAPI구조를 **manage.py**를 통해 **runserver**하는 방식으로 변경

db객체 마이그레이션하면서 에러를 해결하며 계속해서 makemigrations반복.

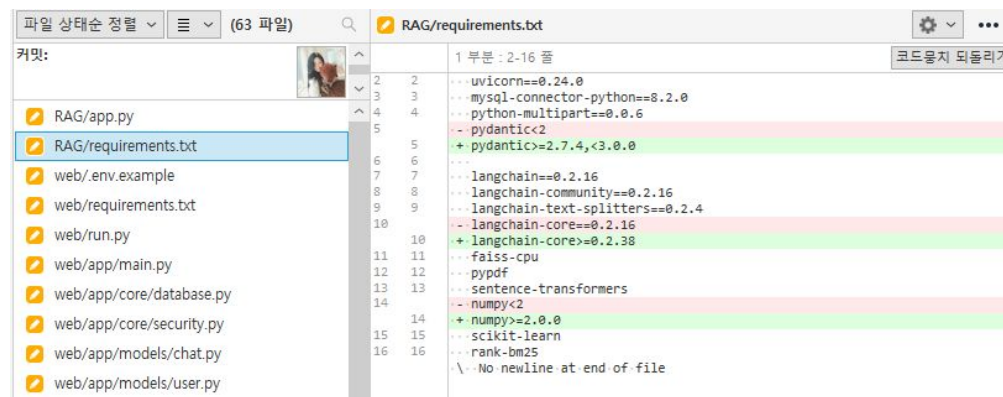
```
python manage.py makemigrations
python manage.py migrate
```

requirements.txt 호환성 문제.

연결된 패키지가 많은 RAG 파트 중 라이브러리가 특정 NumPy API를 요구.

Django 환경을 새로 구성하면서 기존 FastAPI 환경과 다른 Python/패키지 의존성 조합이 설치되
기존 RAG 환경에 잠재되어 있던 dependency compatibility 문제가 드러남.

파일 버전을 맞춰가며 수정완료.



02 Django Migration

Django 마이그레이션

파트: FastAPI프로젝트의 Django 마이그레이션

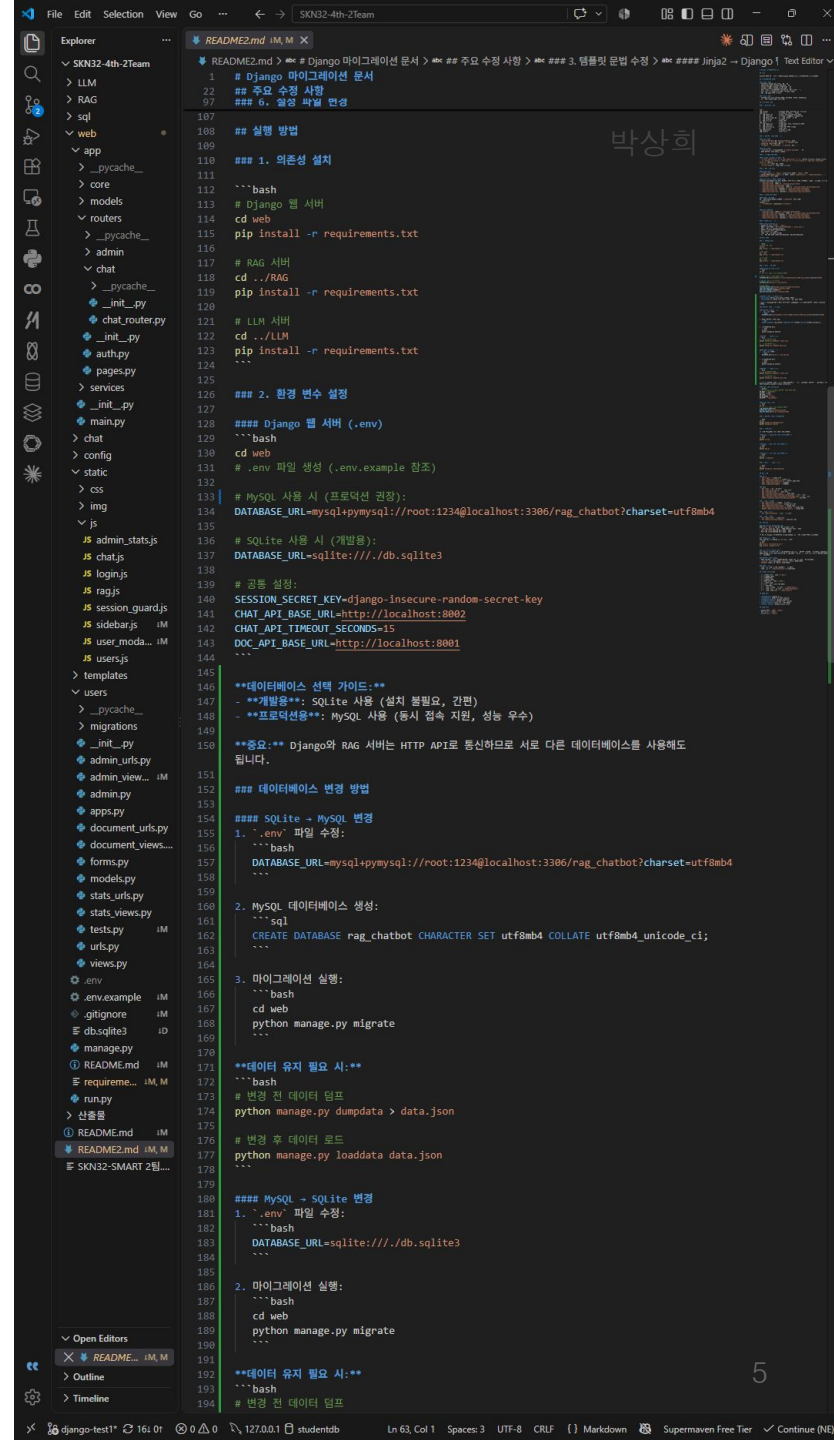
수행 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

완료.

README2.md 파일 생성

마이그레이션에 관한 간략한설명과 실행방법 및 사용될 명령어, 겪었던 에러내역 공유

1. 개요
2. 마이그레이션 범위
3. 포함된 작업과 제외된 작업
4. 수정 후 프로젝트 구조
5. 데이터베이스 모델 변경내역 (User, Chat)
6. 템플릿 문법 수정 내역 (Jinja2)
7. HTTP 메서드 기반 라우팅 명시
8. JavaScript 수정내역 (CSRF 토큰 추가)
9. 설정 파일 변경명시 (config/settings.py)
10. 실행 방법 (의존성 설치, 환경 변수 설정 env)
11. 개발용/배포용 데이터베이스 변경 방법과 기존 데이터 덤프생성과 재로드 명령어 첨부
12. 데이터베이스 마이그레이션 및 각 서버 실행 명령어 첨부
13. 겪었던 에러 사항 기록
 - 503 Service Unavailable 에러
 - 의존성 충돌 해결



모델 운용 구조

운영 API는 provider registry로 OpenAI·Gemini를 전환하고, Qwen은 연구 트랙으로 분리

provider registry 구조 (app/providers/registry.py)

provider별 인스턴스를 한 번 생성해 캐싱·재사용한다. 운영 API는 openai와 gemini를 지원하며, Qwen은 qwen-sft 연구 코드로 별도 관리한다.

운영 기본

OpenAI · gpt-4o-mini

- 답변 생성·주제 분류·의미 검증에 사용
- 초기 34문항 주제 분류 33/34(97%) · 한국어 이탈 0건
- p50 1.9s · p95 2.9s

연구 트랙

Qwen2.5:7b

- 운영 registry에는 미연결
- 초기 주제 분류 32/34(94%) · 한국어 이탈 3건
- 로컬 추론·SFT 연구 후보로 검토

운영 대안

Gemini · gemini-3.5-flash

- 운영 registry가 지원하는 대안 provider
- 비교 실험(Flash-lite) 32/34(94%) · p50 1.4s
- 설정 모델이 달라 전환 전 재검증 필요

프롬프트 엔지니어링 — 실측 기반 튜닝

초기 34문항 실험에서 발견한 오류를 app/prompts.py에 반영

카테고리 정의 구체화

“복무” 포함 질문이 한 범주로 쏠린 오류를 발견. 각 카테고리의 적용 범위를 문장으로 정의해 30/34 → 33/34로 개선.

출력 언어 지시 위치 이동

한국어 지시를 첫 문단으로 이동해 중국어 이탈 5건 → 3건. 완전히 해결되지 않아 Qwen은 답변 생성에서 제외.

예시 수치 제거

환각 방지

“최대 25일” 예시를 실제 규정처럼 복제. 예시 숫자를 제거하고 수치·기간·비율은 검색 근거에서만 사용하도록 제한.

습관적 안내 문구 제한

신뢰도 개선

근거가 있어도 ‘인사팀에 확인하세요’를 반복. 모호하거나 최종 판단이 필요한 경우에만 허용하도록 제한.

답변 생성·검증 계약 구조

검색 근거를 [E1] 단위로 전달하고, 생성 결과를 서버에서 재검증

- ①

출력 언어·역할

한국어 존댓말의 HR 규정 안내 AI. 근거에 없는 조건·사실은 임의로 보완하지 않음
- ②

근거 ID 계약

검색 청크를 [E1]…[En]으로 전달하고, 사실 문장마다 근거 ID를 표기
- ③

적용 기준 보존

사내 규정 우선. 직군·예외·기간·표 범위를 메타데이터와 함께 전달
- ④

답변 규칙 9개

근거 밖 사실·수치 금지 · 모호한 직군·조건은 먼저 확인 · 단순 질문은 1~2문장
- ⑤

4개 응답 상태

ANSWER · CLARIFY · NOT_FOUND · RAG_UNAVAILABLE — 무근거와 검색 장애를 구분

생성 후 [E번호]·수치·조문 검사 → 위험 질문 의미 검증 → 1회 교정 → 실패 시 차단

RAG + LLM 파이프라인 비교

초기 Git 구조 → 현재 구조: 중복 연산·후보 손실 감소 + 답변 검증



항목	초기 Git 기준	현재 Git 기준
검색 인덱스	요청마다 문서별 FAISS 로드	문서별 캐시 + 서버 시작 warmup
질문 임베딩	스토어마다 질문 재임베딩	요청당 1회 임베딩 후 재사용
후보·리랭킹	dense top-20 뒤 BM25·CrossEncoder	문서별 top-3 전체 BM25 → dense·lexical 후보 20 → CrossEncoder top-5
청킹	문자 수 중심 고정 청킹	400/80 + 조문·표 경계·직군·조문·페이지 메타데이터
답변 안전성	프롬프트 준수에 의존	[E번호]·수치·조건 검사 + 위험 질문 의미 검증

RAG 검색 최적화 과정

서로 다른 통제 실험의 동일 조건 전·후 비교 — RAG/bench/REPORT.md

1

질문 임베딩 1회로 통합

스토어마다 반복하던 질문 재임베딩을 요청당 1회로 축소

935.8 ms → **407.6 ms**

2.3×↓ (26 PDF 기준)

2

문서별 FAISS 캐시 + mtime 무효화

요청마다 발생하던 인덱스 재로딩 제거

1,468 ms → **516 ms**

재로딩 비용↓ (832 반복-store 모사)

3

CrossEncoder CUDA FP16 비교 실험

로컬 GPU에서 CPU FP32 대비 리랭킹 병목 가속

5,222.9 ms → **177.5 ms**

29.42×↓ (93 PDF)

품질 유지 expected-PDF hit@5 90.6% (29/32) · MRR 0.683 · CPU/GPU top-5 순위 차이 0건

※ 별도 실험: 832는 반복-store 모사, 177.5ms는 93 PDF warm retrieval(HTTP·Django·LLM 제외). 현재 배포는 CPU Torch.

답변 개선 사례

출근 시간은 몇시야?

출근 시간은 유연근무제에 따라 여러 유형으로 나뉘어 있습니다. 다음은 시차출퇴근 근무시간의 유형입니다:

1. A형: 07:30 ~ 16:30 (7시 30분 출근형)
2. B형: 08:00 ~ 17:00 (8시 출근형)
3. C형: 08:30 ~ 17:30 (8시 30분 출근형)
4. D형: 09:30 ~ 18:30 (9시 30분 출근형)
5. E형: 10:00 ~ 19:00 (10시 출근형)

이는 유연근무제 운영지침 제17조(유형)에 따른 내용입니다.

근거 문서 3건 · 유연근무제 운영지침 p.5, 시간선택제 직원에 관한 ...

휴가 사용일수는 총 며칠이야?

휴가 사용일수는 총 25일을 한도로 하며, 초과하는 계속 근로 연수 매 2년에 대하여 1일을 가산한 유급휴가를 주어야 합니다. 이는 근로기준법에 따른 내용입니다.

근거 문서 5건 · 5.근로기준법(법률) p.14, 유연근무제 운영지침 p....

변경 전 — 근거 없는 수치 단정



출근 시간은 몇시야?

통상 근무시간은 09시부터 18시까지입니다. 시차출퇴근 근무 시간 유형은 다음과 같습니다.

1. A형: 07:30~16:30
2. B형: 08:00~17:00
3. C형: 08:30~17:30
4. D형: 09:30~18:30
5. E형: 10:00~19:00

따라서 출근 시간은 07:30부터 10:00까지 선택할 수 있습니다.

근거 문서 2건 · 유연근무제 운영지침 · 제17조(유형) · p.6, 복무규정 ...

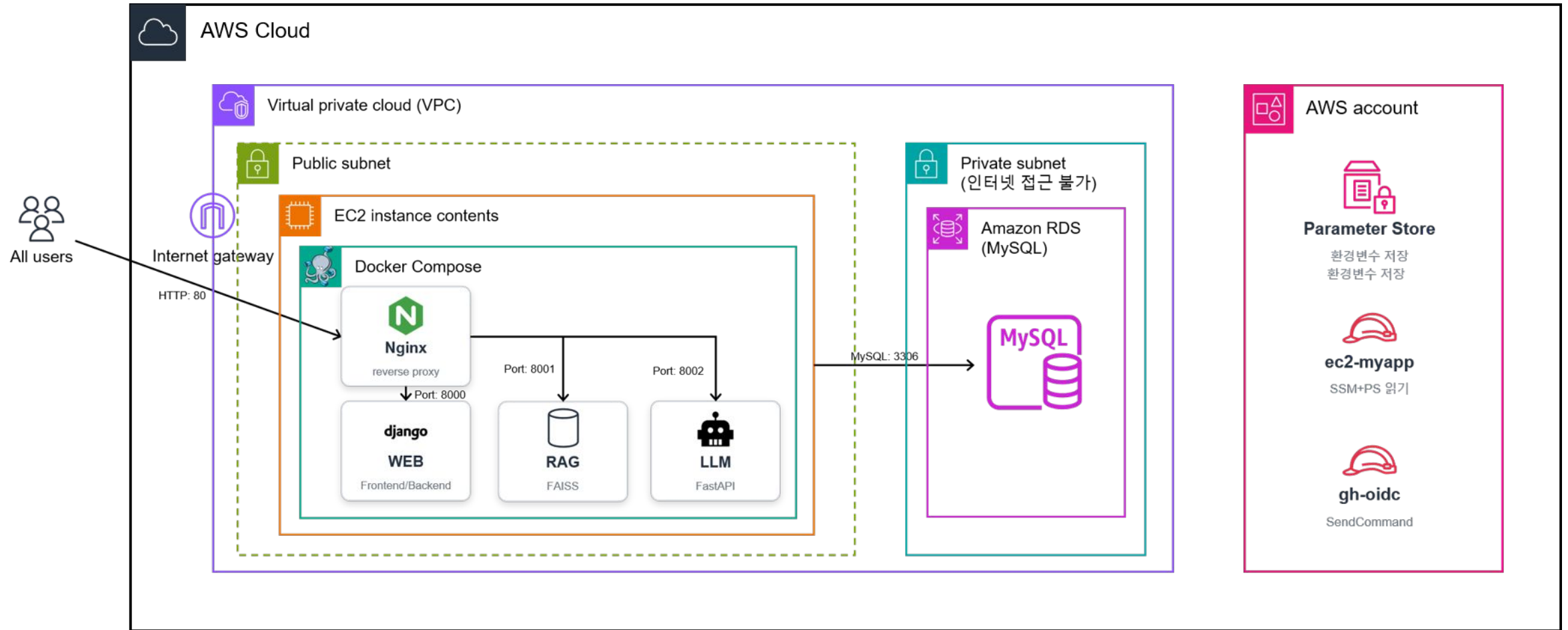
연차 사용 가능일수는 며칠이야?

연차 사용 가능일수는 다음과 같습니다:

1. 계속 재직 기간이 1년 미만인 교직원 또는 1년간 80퍼센트 미만 출근한 교직원: 1개월 개근 시 1일
2. 1년간 80퍼센트 이상 출근한 교직원: 15일
3. 3년 이상 계속 재직한 교직원: 1년을 초과하는 계속근무 연수 2년마다 15일에 1일을 가산하되, 총휴가일수는 25일을 한도로 함.

근거 문서 1건 · 복무규정 · 제20조(연차휴가) · p.4

변경 후 — 대상·조건·출처 명시



CI/CD



타임아웃 설계

- SSM 명령 TimeoutSeconds: 3600s (RAG 최초 벡터스토어 생성 최대 20분 대응)
- WaiterConfig: Delay 15s × MaxAttempts 240 = 3600s (명령 타임아웃과 일치)
- get-command-invocation의 실제 Status로 최종 성공/실패 판정 (waiter 실패는 무시)

AWS 월 예상 운영 비용

ap-northeast-2(서울), On-Demand 기준 — 24시간×30일 상시 운영 가정

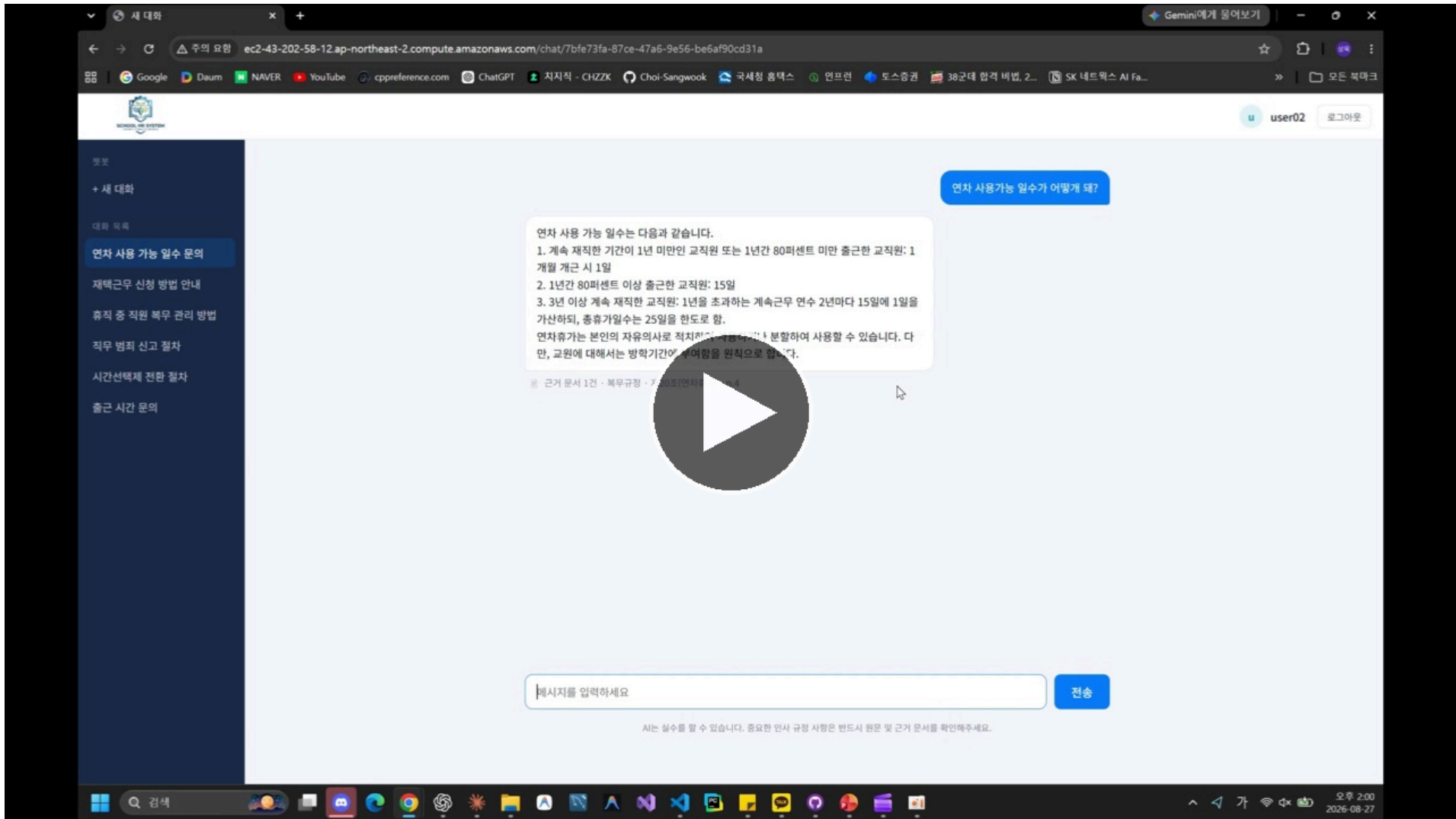
리소스	사양	단가	월 예상 비용
EC2	m7i-flex.large (2vCPU/8GB)	\$0.1177/hr × 730h	\$85.93
EBS	gp3 30GB (EC2 루트 볼륨)	\$0.10/GB-month	\$3.00
RDS 컴퓨팅	db.t4g.micro, Single-AZ MySQL	약 \$0.021/hr × 730h	약 \$15.33
RDS 스토리지	gp3 20GB	\$0.115/GB-month	\$2.30
Parameter Store	Standard 파라미터	무료	\$0.00
데이터 전송(아웃)	월 100GB까지	프리티어 무료	\$0.00 (초과분 GB당 약 \$0.126)
월 합계 (상시 운영 기준)			약 \$106.6 / 월 (약 14~15만원)

※ GitHub Actions(무료 티어), 외부 LLM API(OpenAI/Gemini) 사용료는 AWS 비용에 포함되지 않음 · 가격은 검색 시점(2026-08) 공개 자료 기준 근사치이며 실제 청구액은 AWS Billing Console에서 확인 필요

향후 과제

- 1 도메인 연결 + Nginx/Certbot HTTPS 적용
- 2 무중단 배포(Blue/Green) 구조 도입
- 3 로컬→RDS 직접 bootstrap, FAISS 인덱스 S3 캐싱
- 4 Rollback 워크플로우 (커밋 SHA 기준 재배포)
- 5 트래픽 증가 시 ECS/Fargate 오케스트레이션 전환

시연 영상



감사합니다